MAPLESURE INSURANCE · AMS CONSOLE

S3 — Run Steps

Standing the applications up from a clean checkout, in any sandbox

Generated from `demo/DEMO_STEPS.md` · repo `ams-s3-demo`
Companion docs: `apps/README.md` (what each application is) · `demo/DEMO_TEST_GUIDE.md` (per-scenario rehearsal script)

How to stand this up from a clean checkout, in any sandbox. Written for someone who has never run it before — if you already know the repo, you want `demo/DEMO_TEST_GUIDE.md` instead, which is the per-scenario rehearsal script.

1. What you are standing up

Five processes, each started by its own script under `apps/`, plus the S3 tooling that drives them. You do **not** need all five for every beat.

#	PROCESS	START	PORT	NEEDED FOR
1	**Console** — FastAPI + React. The screen you present from.	`apps/run-console.sh`	8000 + 5173	Every beat
2	**PolicyCore** — the client's plan-administration portal (Streamlit). The window the audience watches change.	`apps/run-policycore.sh`	8501 (open `/sl_policycore`)	CR-2026-041, CR-2026-042
3	**Policy-Service** — ClaimsPortal contracts side (Python/FastAPI).	`apps/run-policy-service.sh`	8081	CR-2026-043
4	**Claims-Service** — ClaimsPortal claims side (Python/FastAPI). Start after #3.	`apps/run-claims-service.sh`	8082	CR-2026-043
5	**EnrolDirect** — the online enrolment channel (Python/FastAPI).	`apps/run-enroldirect.sh`	8083	CR-2026-045

> OPEN THIS ONE
> The console UI is `http://localhost:5173`, not `:8000`. Port 8000 is the API the UI talks to.

Two folders, and the difference matters

	HOLDS	S3'S RELATIONSHIP TO IT
`repos/`	The target repositories — `policycore/`, `claimsportal/`, `enroldirect/`	Something S3 **changes**
`apps/`	The console (`apps/console/`) and the five launch scripts above	Something that **does the changing**

Python imports follow the folders: the targets are `repos.policycore.*`, `repos.claimsportal.*`, `repos.enroldirect.*`; the console is `apps.console.api.main:app`.

`repos/` is also a drop folder. A directory placed there with a `.s3targets.json` manifest registers itself as a target at next start — no edit to `s3_enhancement/targets.py`. See **`repos/README.md`** for the manifest contract and the onboarding steps; `apps/README.md` covers the launch scripts and how each process maps to a ServiceNow application.

---

2. Prerequisites

TOOL	VERSION	NEEDED FOR	CHECK
Python	3.12+	Console API, PolicyCore, S3 tooling, ClaimsPortal	`python3 --version`
Node.js	18+	Console UI	`node --version`

---

3. First-time setup from base packages

From the repository root. Run once per machine.

```
# 1. Python environment
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 2. Console UI dependencies
cd apps/console/web && npm install && cd ../../..

# 3. Configuration
cp .env.example .env
#    then edit .env — see section 4

# 4. Seed the PolicyCore database
python -m repos.policycore.core.seed

# 5. Confirm the install
python -m pytest -q          # expect: 679 passed
```

> **SANITY CHECK**
>
> If `pytest` passes, the wiring is correct. It exercises the routing registry, the code-review flow and all four replay targets without needing an LLM or any of the five processes running.

---

4. Pointing it at your LLM

Edit `.env`. Pick *one* provider block.

Self-hosted / custom model (most sandboxes)

For any endpoint that speaks the OpenAI chat API — an internal gateway, vLLM, LiteLLM, TGI, LM Studio:

```
LLM_PROVIDER=custom
CUSTOM_LLM_BASE_URL=https://llm.internal.example/v1
CUSTOM_LLM_MODEL=llama-3.3-70b
CUSTOM_LLM_API_KEY=            # optional — blank is fine if your gateway does no auth
```

> **THE BASE URL IS USED EXACTLY AS WRITTEN**
>
> Include the version segment your gateway expects, usually `/v1`. Nothing is appended or rewritten, so a wrong URL gives a clean 404 rather than a confusing one. The client requests `<CUSTOM_LLM_BASE_URL>/chat/completions`.

Verify the endpoint is reachable before the session:

```
python -c "
from common.llm import complete
print(complete('Reply with the single word: ready'))
"
```

Other providers

PROVIDER	`LLM_PROVIDER`	ALSO SET
Anthropic	`anthropic`	`ANTHROPIC_API_KEY`, `ANTHROPIC_MODEL`
OpenAI	`openai`	`OPENAI_API_KEY`, `OPENAI_MODEL`
Claude via Bedrock	`bedrock`	`AWS_REGION`, `BEDROCK_MODEL`
Ollama (local)	`ollama`	`OLLAMA_BASE_URL`, `OLLAMA_MODEL`

Replay mode — why the run survives a bad network

The default is `LLM_MODE=replay`. The code-generation and test-generation beats are served entirely from committed recordings in `s3_enhancement/cache/` and never call your model at all. That is what makes the run deterministic.

> **SHARP EDGE — READ THIS ONE.**
>
> The *narrative* beats (impact analysis, effort estimate, release notes, design doc) go through a different cache that does **not** honour `LLM_MODE`. On a cache miss they call your endpoint live, even in replay mode. Always run `demo/warm_s3_cache.sh` before presenting — otherwise a slow or unreachable model stalls a beat mid-run.

---

5. Reset before every rehearsal or session

The pipeline mutates real source files and the SQLite database. Reset returns everything to the pre-CR baseline. Run them **in this order** — the amendment reset must come after `reset_s3.sh`, because that script reseeds the database the amendment baseline then builds on.

```
demo/reset_s3.sh                # CR-2026-041 (PolicyCore plan tier) + shared state
demo/reset_s3_endorsement.sh    # CR-2026-042 (PolicyCore amendment priority)
demo/reset_s3_claimsportal.sh   # CR-2026-043 (ClaimsPortal deductible)
demo/reset_s3_enroldirect.sh    # CR-2026-045 (EnrolDirect prospect access)
demo/warm_s3_cache.sh           # ALWAYS last — reset_s3.sh wipes .cache/llm
```

Expected final lines:

```
S3 source baseline restored, PolicyCore reseeded, LLM cache cleared, and ticket timeline cleared.
CR-2026-042 source baseline restored, PolicyCore reseeded, and LLM cache cleared.
```

```
ClaimsPortal source baseline restored; generated files removed.
EnrolDirect source baseline restored; generated files removed.
```

The first two restore from `HEAD`

`reset_s3.sh` and `reset_s3_endorsement.sh` restore PolicyCore with `git checkout HEAD --repos/policycore/...`, so **HEAD has to contain those paths**. Move a target directory and both stop at the checkout —

```
error: pathspec 'repos/policycore/app.py' did not match any file(s) known to git
```

— before the reseed, the `.cache/llm` wipe and the ticket-timeline clear. **Committing the move is the fix**; nothing in the scripts needs changing. (This is not hypothetical: it happened on 2026-08-03 while the `apps/` → `repos/` move was uncommitted, and commit `e5af8ed` cleared it.) Check it read-only whenever a target has just moved:

```
git cat-file -e HEAD:repos/policycore/app.py 2>/dev/null && echo "in HEAD" || echo "NOT in HEAD"
```

- **`reset_s3_claimsportal.sh` and `reset_s3_enroldirect.sh` never touch git** — they restore by `cp` from the in-repo `.baseline/` snapshots, so a target move cannot affect them.
- The admin panel checks the same thing up front: `GET /api/admin/status` carries a `reset_blocked_reason` naming any path missing from HEAD, and disables the PolicyCore reset with that reason on it rather than running a script that would fail halfway.

Either way, confirm a reset with the baseline checks in `DEMO_TEST_GUIDE.md` section 0a rather than assuming it worked — watch all four scripts print their success line.

---

6. Start the applications

One terminal per process. For the PolicyCore CRs you need only the first two.

```
apps/run-console.sh           # terminal 1 — API :8000 + UI :5173
apps/run-policycore.sh        # terminal 2 — portal :8501/sl_policycore

# only for the ClaimsPortal CR (CR-2026-043):
apps/run-policy-service.sh    # terminal 3 — :8081  (start before claims)
apps/run-claims-service.sh    # terminal 4 — :8082

# only for the EnrolDirect CR (CR-2026-045):
apps/run-enroldirect.sh       # terminal 5 — :8083
```

Then open `http://localhost:5173` and log in. Any roster name works; passcodes are `1001 + position in the roster`, and `Manager` is fixed at 9000 — the scheme lives in `common/roster.py`.

LOGIN	PASSCODE	ROLE
Ravi Kumar	1001	Engineer — App Support, PolicyCore
Elena Cruz	1002	Engineer — App Support, PolicyCore
Priya Nair	1003	**Tester** — the QA hand-off receives work here
Tom Becker	1004	**Tester** — the second QA login

LOGIN	PASSCODE	ROLE
Arjun Mehta	1005	Engineer — App Support, ClaimsPortal
Clara Bishop	1006	Engineer — App Support, ClaimsPortal
Manager	9000	Manager view — **the only login that can assign tickets or open `/admin`**

Each role sees only its own stages. This is why the same pipeline looks different depending on who logs in — it declutters each person's view, and it is presentation only (the API does not enforce it; the map is `apps/console/web/src/pages/s3/stageAccess.ts`).

ROLE	STAGES
Manager	Board, `/admin`
Engineer	Board, Target selection, Generate the change, Draft design doc
Tester	Board (their QA queue), Draft design doc, Generate tests + run, Draft release notes

The tester has no Generate stage on purpose: a tester who can regenerate the change under test is not an independent check of it. The manager routes the work and stops there — generated code and a test run are the engineer's and the tester's evidence, and the release note is written by whoever produced the evidence it cites rather than handed back up to be written from a summary. The work never returns to the manager.

6b. What is on the board when you log in

Six tickets. Five are seeded; the sixth opens by itself.

KEY	SUMMARY	STATUS	ASSIGNEE	WHERE IT COMES FROM
AMS-101	CR-2026-041: Plan Tier Upgrade Option	QA	Priya Nair	Seeded Jira replay cache
AMS-102	CR-2026-042: Amendment Priority Field	In Progress	Ravi Kumar	Seeded Jira replay cache
AMS-103	CR-2026-043: Claims Deductible Handling (ClaimsPortal)	To Do	Ravi Kumar	Seeded Jira replay cache
AMS-098	Quarterly policy data cleanup	Done	Elena Cruz	Seeded Jira replay cache (background noise)
AMS-104	Flag urgent amendment requests (from Support Ops)	To Do	Ravi Kumar	`demo/seed_s3_repo_selection_ticket.sh` (CR-2026-044)
AMS-1045	**CR-2026-045: Prospect Member Eligibility Check For Online Enrolment**	**To Do**	**— unassigned —**	**Opened automatically from `crs/CR-2026-045.md`**

Statuses are the *seeded* ones; the board overlays whatever a run has since moved them to, and `reset_s3.sh` restores the seeded set — except that it cannot run today (section 5), so expect to see

wherever the last rehearsal left them until the `repos/` move is committed. `git checkout -- 's3_enhancement/cache/jira_*.json'` plus `rm -f data/ticket_events.jsonl` is the manual equivalent.

AMS-1045 is the beat worth showing. Nobody seeded it. Dropping a CR file into the top-level `crs/` opens a ticket for it, keyed deterministically off the CR id (`CR-2026-045` → `AMS-1045`, in the AMS-1000+ band so it can never collide with the hand-seeded AMS-100..999 tickets), and it lands **unassigned** — which is exactly what puts it in front of the manager on the dashboard, with an Assign control next to it. See `s3_enhancement/cr_intake.py`.

Pair it with the reassignment beat: log in as **Manager / 9000**, assign AMS-1045 to an engineer, then press **Reassign** on the row and change your mind. Assignment used to be set-once; it is now assign / reassign / unassign, and `POST /s3/jira/assign-ticket` is **manager-only enforced server-side** (`require_manager` in `apps/console/api/auth.py`), not merely hidden in the UI. Log in as an engineer and the control is gone; the endpoint would refuse it anyway.

---

7. Optional — seed the routing beat

With the console API already running, this creates a ticket that carries a ServiceNow Configuration Item, so the console can route it to an owning team before any AI step runs.

```
# Default: routes to BillingGateway — an application with an owning team and
# NO repo here. Routing succeeds; automation correctly stays off.
demo/seed_problem_record_ticket.sh

# The other half: routes to a team AND offers the CR to run against it.
SEED_CI=ClaimsPortal SEED_BUSINESS_SERVICE="Claims Management" \
  demo/seed_problem_record_ticket.sh
```

Re-running with a different `SEED_CI` re-routes the same ticket; you do not need to reset between the two.

---

8. The walkthrough flow

The console is six stages on a left-hand rail — **Board → Target selection → Generate the change → Draft design doc (for QA) → Generate tests + run → Draft release notes** — each on its own URL under `/s3/`. The rail is unchanged; what changed is what sits *inside* a stage.

> **ARTIFACTS OPEN IN A POP-UP NOW, NOT INLINE.**
>
> Release notes, the deployment plan, the design doc, the scenario table, the test checklists, the mutation diff and the traceability matrix used to stack down the page. Each stage now shows the action button, a one-line verdict, a compact summary chip proving the artifact exists, and a **View…** button that opens the body in a modal. Action buttons, verdict lines and the token-cost lines stay in the main flow.
>
> This changes the click-path, so re-learn it before presenting: **do not say "scroll down to see the release notes"** — press the View button, read, press Escape or the close button. The modal covers the stage rail on purpose, so nobody navigates away mid-read. Driven by the client's "there is a lot of data on the screen… open it in a pop-up" feedback.

#	BEAT	WHAT TO SAY IT PROVES
0	The board itself — AMS-1045 sitting there unassigned	Onboarding a change is dropping its CR file in. The ticket opened itself, and landed unassigned so a manager routes it

#	BEAT	WHAT TO SAY IT PROVES
0b	As **Manager / 9000**: assign AMS-1045, then **Reassign** it	Assignment is a manager decision, reversible, and enforced server-side — an engineer's session cannot call the endpoint at all
1	Open the ticket; routing panel appears above the analysis	The CI resolved to an application, owning team and repo by table lookup — no model call, nothing to confirm
2	Impact analysis + effort estimate	Vague tickets get a clarifying question first, rather than a confident guess
3	Generate code	Only the files the relevance funnel selected are sent — the token panel shows scoped vs naive cost
4	**Review file by file**: Ask, Apply this file, Reject	Developers accept or reject one file at a time; a rejection records a reason to the ticket's audit trail and is excluded from Apply
5	Apply, then look at PolicyCore on :8501/sl_policycore	The client's running application changed — on CR-2026-041 the new plan tier is selectable and the monthly contribution recalculates
5b	**Source control panel**: branch → commit → push	The change lands on a feature branch cut off `main` before anything is written, the commit is gated on the tests passing, and the push hands off to the pipeline — the flow, not a direct edit to main
6	**Revert** (per file or all)	Anything applied can be undone without a full reset
6b	**Design doc: change map + Download PDF**	The hand-off document carries a diagram of what the change touches, and leaves as a real PDF you can attach to the ticket
7	**Draft test scenarios**, edit one, approve the plan	QA reviews *what will be checked*, in prose traced to the CR's acceptance criteria, before any test code exists — and can change it
8	Generate tests, run them, then the seeded-bug check	The generated tests actually catch a deliberately introduced bug
9	**Run the regression suite**	A human-authored suite the AI cannot write to still passes — the CR cost nothing that already worked
10	**Build the traceability matrix**	Every acceptance criterion, the scenarios planned for it, the tests that ran, and the result — the artifact an auditor asks for
11	Design-doc drift check	Documentation drift is detected automatically after Apply, not by remembering to press a button
12	**Release notes — three audiences** + the derived **deployment & rollback plan**	One note per reader (client / ops / user guide), and a deploy order computed from the change's own service graph
13	**Download the release record, attach it to the ticket**	Everything the run proved, in one PDF — including what it could *not* prove
14	*(if asked "how do you reset between runs?")* `/admin`, as Manager	The product's own housekeeping is in the product, gated, and honest about what it will not do — see below

Beat 14 is the admin panel (`http://localhost:5173/admin`, manager only). Four cards: **Reset environment state**, **Target applications**, **Logs**, **Onboard a repo**. It is worth showing precisely because of what it refuses to do:

- Source-restoring resets **409 while the tree is dirty**, and preview exactly what they would restore, what they would delete, and which of those files currently carry uncommitted changes — before you press anything.
- There is **no "reset everything"** button. Seven explicit scopes: PolicyCore, ClaimsPortal, EnrolDirect, tickets, logs, proposals, caches.
- **No service id for the console itself** — it cannot restart the process serving the request, and does not pretend it can.
- Service status is a plain TCP port probe (no `ps`, no `lsof`), so it works on a locked-down host. "Up" means something is listening on the port, nothing more.
- **Onboard a repo** validates a `repos/<name>/.s3targets.json` and can write it, but a written manifest **needs a console restart** to take effect — discovery runs at import. The panel says so.

All four reset scopes run today. When one *is* refused — a dirty tree, or a path missing from HEAD after an uncommitted target move (section 5) — the button carries a `reset_blocked_reason` saying which, instead of firing a script that would fail halfway. That is the gate working, and it is a fine thing to say out loud.

On beats 12-13: the deployment order is **derived**, not drafted — on CR-2026-043 the plan puts policy_service before claims_service because claims_service calls it, and says why. The release record is assembled from what the run actually produced; its "Not evidenced by this release" block is the part worth pausing on, because a release document that only lists successes is marketing. **Attach to ticket** is honest about the default: with `JIRA_MODE=replay` there is no Jira to attach to, so the beat records the intent on the ticket timeline and says the upload was simulated. Set `JIRA_MODE=live` and it uploads for real.

On beat 5b: say plainly that the git flow is **modelled, not executed** — the panel says so on screen and the release record repeats it under "Not evidenced by this release". Nothing runs git and no remote is contacted. That is deliberate: the target apps live inside this repo and the reset scripts restore the baseline from `HEAD`, so a real commit would make them start restoring the CR instead. The point of the beat is the *shape* of the flow — branch before edit, commit gated on green tests, pipeline on push — which is what a reviewer asks about when they see an AI editing code.

Two things are worth clicking rather than describing. Press **Commit to branch** before running the tests: it refuses, and names the reason, because the gate is computed server-side from the ticket's own test results — the console cannot assert "tests passed". And open **What a real integration would have run** for the git transcript, which grows one step at a time as you take each step. If you Revert everything afterwards, the branch shows as *abandoned* rather than disappearing, and a commit already made is not unmade — in a real repo the honest undo at that point is a revert commit, not a rewritten history.

On beat 6b: the change map is **derived, not drawn by the model** — services, layers and the cross-service arrow are read from the changed-file set, so it costs no LLM call and needs no cache warming. The `NEW` badge comes from git (the file is absent from `HEAD`), which is why `claim_rules.py` carries one on CR-2026-043 and nothing does on CR-2026-042. The PDF is rendered server-side by headless Chromium; if `playwright install chromium` has not been run on the presenter machine the endpoint answers 503 and the console silently falls back to the browser's own print-to-PDF, so the button always does something.

Beats 7, 9 and 10 are the QA-facing half of the tests stage. Two things worth saying out loud when showing them:

- The regression suite (`tests/test_regression_policycore.py`, `tests/test_regression_claimsportal.py`, `tests/test_regression_enroldirect.py`) appears in **no**

target's `testgen_allowlist`. The pipeline physically cannot write to it, which is what makes "the pre-existing tests still pass" a result rather than a claim.

- In the matrix, only the scenario→test column is inferred, and it is deliberately conservative: an ambiguous pairing renders as "no automated test" rather than guessing. On CR-2026-043 two criteria legitimately land there — that is the honest answer, and a good moment to make the point that the tool reports gaps instead of hiding them.

For the full per-scenario talk track and the fallback ladder, see `demo/DEMO_TEST_GUIDE.md`.

---

9. Troubleshooting

SYMPTOM	CAUSE AND FIX
`error: pathspec 'repos/policycore/app.py' did not match any file(s) known to git`	`reset_s3.sh` / `reset_s3_endorsement.sh` restore from HEAD, and a target directory was moved without committing the move, so HEAD does not have that path. **Commit the move** — see section 5. The ClaimsPortal and EnrolDirect resets restore from `.baseline/` and are unaffected.
`FOREIGN KEY constraint failed` during a reset or seed	An old baseline whose `wipe_db()` predates the amendments table (it was called `endorsements` before the 2026-08-03 GRS reskin). Fixed in the current scripts. If you hit it on an older checkout: `rm -f data/mockapp.db`, then re-run the reset.
`codegen returned unexpected file set`	A target directory under `repos/` was renamed or moved. `relevance.py` folds each file's path into the text it scores, so a rename reshuffles the selection and desyncs it from the committed recordings. Restore the directory name — see `repos/README.md` and the "File paths are load-bearing" section of the root `CLAUDE.md`.
A reset button in `/admin` is greyed out	Deliberate. Hover it — `reset_blocked_reason` says whether it is the dirty tree or a path missing from HEAD. Source resets never run over uncommitted work.
A newly onboarded repo does not appear as a target	The manifest is only read at import. Restart the console API.
A beat hangs, or fails mentioning your LLM URL	A narrative beat missed its cache and called your model. Run `demo/warm_s3_cache.sh`. Confirm the endpoint with the one-liner in section 4.
Applied change crashed the portal	Expected and handled — the console shows the migration traceback in a pop-up with a one-click fix. You can also press **Revert all**.
Console UI loads but every call 401s	Not logged in, or the API on :8000 is not running. Check terminal 1.
`/admin` 403s, or the Assign control is missing	You are logged in as an engineer. Both are manager-only, enforced server-side. Log in as **Manager / 9000**.
Claims-Service errors on a claim	Policy-Service on :8081 is not up. Claims calls policy over HTTP; start #3 first.

SYMPTOM	CAUSE AND FIX
Port already in use	`lsof -ti:8000 | xargs kill` (repeat per port). Note that `--reload` is deliberately not used on the API: the reloader restarts mid-beat when codegen writes to the tree. Also note `demo/run_s3.sh` (the legacy Streamlit console) defaults to :8501 and will collide with PolicyCore.

---

10. Pre-session checklist

- ☐ `python -m pytest -q` passes (679 tests)
- ☐ `.env` has your provider block filled in, and the section-4 one-liner returned a reply
- ☐ The `repos/` move is committed — otherwise the two PolicyCore resets cannot run (section 5)
- ☐ All four reset scripts ran clean, in order, and each printed its success line
- ☐ `demo/warm_s3_cache.sh` ran *after* the resets
- ☐ Console reachable at `:5173`; you are logged in
- ☐ PolicyCore reachable at `:8501/sl_policycore` in a second window
- ☐ For the ClaimsPortal CR: `:8081` and `:8082` both responding
- ☐ For the EnrolDirect CR: `:8083` responding
- ☐ AMS-1045 is on the board and unassigned
- ☐ You have opened and closed at least one artifact modal, so the click-path is muscle memory
- ☐ You have walked beats 0–13 once, end to end, on this machine

---

The target repositories live under `repos/`; the console and the launch scripts under `apps/`; the AI pipeline that drives them in `s3_enhancement/`, shared clients in `common/`, and presenter scripts in `demo/`. Do not rename directories under `repos/` — the paths are a scoring input to file selection and are baked into the committed replay recordings. See `repos/README.md`.